Vocard API documentation

Textbook ingestion. Stateless grammar answers. Vocabulary tutoring.

Developer reference for vocard.api:create_app and the separate vocard.chat:create_chat_app playground. Updated 17 October 2026 against repository routes, request models and privacy controls. This describes implementation behavior, not a verified live deployment.

Document scope

The production API uses Firebase authentication for student questions and separate administrator JWTs for ingestion, retrieval and review. Private PDFs, book versions, indexes and operational metadata remain durable. Student questions, answers and temporary tutor context are not persisted by the application.

Changes in this edition

- Added POST /ai/vocabulary/explain, contextual vocabulary meanings, ambiguity handling and explicit source labels.
- Removed conversation IDs, history storage, conversation-loading endpoints, request leases and history replay.
- Made course, unit and lesson context optional for grammar requests.
- Added bounded client-supplied prior_explanation only to vocabulary tutoring; it is untrusted temporary context.
- Documented no-store responses, content-free quotas, legacy cleanup and provider retention separately.
- Updated the standalone playground to accept one question instead of message arrays.

Reading guide

02 Endpoint directory; 03 Authentication and startup; 04 Grammar request; 05 Grammar response; 06 Vocabulary request; 07 Vocabulary response; 08 Sources and follow-ups; 09 Privacy; 10 Administrator retrieval; 11 Ingestion; 12 Jobs and review; 13 Errors; 14 Playground; 15 Configuration and source map.

Base URLs are deployment-specific HTTPS gateway URLs. Local production development defaults to http://127.0.0.1:8000; the separate playground defaults to port 8001. JSON examples are illustrative, not saved student interactions.

Endpoint directory

Method and path	Purpose / success
GET /healthz	Public process health / 200
POST /ai/answer	Firebase: independent grammar question / 200
POST /ai/vocabulary/explain	Firebase: contextual vocabulary tutoring / 200
POST /admin/books/{book_id}/ingestions	Administrator: PDF and optional extraction / 202
GET /admin/books/{book_id}	Administrator: book/version/index history / 200
POST /admin/books/{book_id}/search	Administrator: semantic retrieval / 200
POST /admin/books/{book_id}/answer	Administrator: grounded answer / 200
GET /admin/books/{book_id}/lessons	Administrator: exact lesson lookup / 200
GET /admin/jobs/{job_id}	Administrator: job status / 200
POST /admin/jobs/{job_id}/retry	Administrator: eligible failed-job retry / 202
GET /admin/jobs/{job_id}/extraction	Administrator: extraction JSON download / 200

Conventions

Paths are relative to the configured base URL. JSON requests use Content-Type: application/json; ingestion uses multipart/form-data. Book IDs match [A-Za-z0-9][A-Za-z0-9_-]{0,99}. PDF page indexes are 1-based and distinct from printed page labels. Book/job timestamps are Unix seconds.

Student request models reject unknown fields, client UIDs, arbitrary book IDs, conversation IDs and message arrays. Question parameters belong in JSON bodies, not URL query strings. Q&A errors do not echo rejected inputs. Q&A responses include Cache-Control: no-store and Pragma: no-cache.

The retired GET /ai/conversations and GET /ai/conversations/{conversation_id} routes no longer exist and return 404. They must not be used to restore a timed-out response. No conversation ID is returned by student answering.

Neither app exposes /docs, /redoc or /openapi.json. GET /healthz returns {"status":"ok"}; this is process health, not an end-to-end dependency readiness test. The playground routes on page 14 belong to a separate app.

Authentication and startup

Student API: Firebase ID token

```
Authorization: Bearer <FIREBASE_ID_TOKEN>
Content-Type: application/json
Cache-Control: no-store
```

Firebase Admin verifies the ID token, including revocation, and derives the UID from verified claims. Missing, invalid, expired or revoked tokens return 401; anonymous users return 403. An unavailable or unconfigured verifier returns 503. The client cannot select another UID.

All non-anonymous users in the configured Firebase project may access mapped grammar courses and the eight supported vocabulary courses. There is no per-student enrollment check. Grammar access is controlled by VOCARD_COURSE_BOOKS; vocabulary can use model knowledge without a reference-book mapping.

Administrator API: separate HS256 JWT

Claim	Requirement
sub	Nonblank administrator identity
iat, exp	Required issue and expiry timestamps
iss	VOCARD_JWT_ISSUER; default vocard-admin
aud	VOCARD_JWT_AUDIENCE; default vocard-ingestion
role	Exactly administrator
book_ids	Optional string array; omitted unrestricted, empty denies all books

Use a secret of at least 32 bytes. There is no token-issuance endpoint. Invalid credentials return 401; wrong role or book scope returns 403. Administrator authentication precedes multipart parsing.

Separate API and worker processes

```
.venv/bin/uvicorn vocard.api:create_app --factory \
  --host 127.0.0.1 --port 8000 --no-access-log
.venv/bin/python -m vocard.worker
```

Export configuration first; .env is not loaded automatically. The API checks private S3 configuration at startup. API and workers share durable SQLite on one host/local volume. Remote access requires HTTPS; Flutter web needs same-origin routing or an explicit gateway CORS allowlist. Legacy conversation rows block startup until the explicit privacy migration is completed.

Student grammar question

POST /ai/answer requires a Firebase ID token. Every request is independent.

JSON field	Contract
question	Required nonblank string, 1-4000 characters
course_id	Optional/null nonblank string, 1-100 characters
unit_id	Optional/null nonblank string, 1-100 characters
lesson_number	Optional/null strict integer >= 1; explicit asset lesson number

```
{
  "question": "Explain gerunds as objects.",
  "course_id": "devils_100",
  "unit_id": "devils-grammar-unit-01",
  "lesson_number": 1
}
```

A question-only request is allowed. If the course is omitted, a single unique book in the server mapping may be selected. Multiple mapped books require a focused course clarification; an empty mapping denies access. An explicitly unmapped course returns 403. The client cannot override book, retrieval settings, language or practice-example generation.

Retrieval and missing context

The server retrieves from the authorized book's current active index for every scoped question, using only the current question. Optional course/unit filters narrow the scope. Current-lesson evidence is preferred, with controlled fallback within the same authorized book and scope. No stored questions expand retrieval.

No usable evidence returns insufficient_evidence; unavailable references such as a bare follow-up asking why require a focused clarification. A missing active book index returns 404. An index promotion during answering returns 409 so a fresh request can retrieve the new version.

Client integration

Render the answer, citations and source examples together. Keep displayed messages only in screen memory and discard them on close or account change. Never send conversation_id, history, messages, previous_messages or vocabulary-only prior_explanation to this route; extra fields return 422.

There is no history reload, streaming response or student idempotency key. A timeout can leave an admitted server request running, but its response is not saved. An explicit retry is a new request and may consume another quota unit.

Shared grammar answer contract

Both student grammar answering and administrator answering return the same object, without a conversation ID.

Field	Meaning
answer	Korean explanation by default, abstention text, or one clarification question
status	answered / insufficient_evidence / clarification_needed
citations	Retrieved chunk IDs, exact quotes and server-resolved locations
source_examples	Exact source text, correctness and translations
practice_examples	Opt-in generated examples; administrator answer only

Citation object

```
{
  "chunk_id": "〈retrieved-chunk-id〉",
  "quote": "〈verbatim supporting text〉",
  "book_id": "devils-100", "version": 1,
  "index_version": "〈immutable-index-id〉",
  "source_pages": [15, 16], "source_page_labels": [],
  "course_id": "devils_100",
  "unit_id": "devils-grammar-unit-01", "lesson_number": 1
}
```

Course, unit and lesson metadata may be null. Printed labels are separate from 1-based PDF pages. The server resolves all location metadata. Only chunks included in that request's generation context can be cited. Unknown IDs, changed quotes or changed source examples fail closed with 502. Provenance checks do not independently prove semantic entailment.

Examples and status handling

source_examples items contain chunk_id, text, is_correct (boolean/null), and extraction-provided translations. Preserve punctuation and correct/incorrect markers; never silently repair source examples.

practice_examples contain english, translation and the fixed label 새로 만든 연습 예문 (교재 원문 아님). Show this label with every generated example. They require administrator include_practice_examples=true.

insufficient_evidence is a valid 200 response with empty citations/examples. A generated grammar answer lacking citations also abstains. clarification_needed asks one focused question. Provider failures return sanitized 503 errors. Machine-readable contract: schema/answer-response.schema.json.

Vocabulary tutoring request

POST /ai/vocabulary/explain requires Firebase authentication. It reuses VocabularyWord fields from the Flutter app, including visible example translations from the existing model getter.

```
{
 "question": "What does bank mean in this sentence?",
 "selected_entry": {
  "id": "ielts-bank", "courseId": "ielts", "word": "bank",
  "meaning": "은행; 강둑", "explanation": "문맥에 따라 뜻이 다릅니다.",
  "exampleSentences":["She sat on the bank."],
  "exampleTranslations":["그녀는 강둑에 앉았다."]
 },
 "sentence": "She sat on the bank of the river.",
 "prior_explanation": null
}
```

Field	Contract
question	Required nonblank string, 1-4000 characters
selected_entry	Required object; id, courseId, word, meaning required
sentence	Optional/null nonblank string, 1-2000 characters
prior_explanation	Optional/null nonblank excerpt, 1-2000 characters

Entry limits: id and word at most 200 characters; courseId at most 100; meaning at most 2000; optional explanation at most 4000. Optional exampleSentences and exampleTranslations each contain at most 20 nonblank strings of at most 2000 characters. Total generation context must fit VOCARD_ANSWER_CONTEXT_CHARS. Omit optional empty strings.

Supported course IDs: suneung, toeic, toefl, ielts, gtelp, idioms, suneung_idioms, collocations. Unknown courses return 403. Extra fields, arbitrary book IDs, client UIDs, conversation IDs and previous-message arrays return 422.

Every client field is untrusted context, including the asserted entry identity, meanings, examples, sentence and prior explanation. The API does not verify the entry against a server vocabulary catalog and does not promote client text to system instructions or trusted evidence. Full contract: docs/vocabulary-tutor-api.md; input schema: schema/vocabulary-question.schema.json.

Vocabulary tutoring response

Field	Contract
status	answered / ambiguous / clarification_needed
contextual_meaning	part_of_speech, explanation_ko, source_ids; null for clarification
interpretations	Alternative meaning objects; at least two for ambiguous
nuance	register, connotation, collocations, comparisons, source_ids; nullable
examples	english, translation, source_id, is_correct, server label
suggested_follow_up_questions	1-4 suggested questions, each at most 400 characters
clarification_question	One focused Korean question or null
sources	Server-created source_id, kind, label, citation records

Context comes first

The tutor explains the meaning that fits the current sentence first, with part of speech and a concise Korean explanation. It then covers register, connotation, collocations and relevant comparisons. General linguistic knowledge is allowed, with generated-content attribution rather than invented textbook authority.

If several meanings fit, ambiguous gives at least two likely interpretations and may ask one focused question. If essential context is unavailable, clarification_needed contains a single Korean clarification question, null contextual meaning/nuance and empty interpretations/examples/book citations.

Answered responses must include at least one correct example with a translation. Newly generated examples must have is_correct=true and Korean translations. Source examples can remain incorrect, but must retain their text, translations and incorrect markers. A source example without a supplied translation may return null rather than invent a source translation.

Every contextual-meaning and nuance object includes model in source_ids, even when it draws on app or book data. Book IDs/page claims cannot be invented in prose. Server-created source labels and locations are described on page 08.

Output schema: schema/vocabulary-tutor-response.schema.json. The provider's internal draft contains citation IDs and quotes; public citations are resolved into sources[].citation by the server. No conversation ID is returned and no response is saved as server history.

Vocabulary sources and follow-ups

source_id / kind	Server-owned label and meaning
model / model_generated	AI 생성 설명 · 교재 인용 아님
vocard:entry / vocard_data	Vocard 데이터 (앱 제공 · 서버 미검증)
vocard:example:N / vocard_data	Verbatim app example pair; zero-based array index
book:CHUNK_ID / book_passage	Retrieved book evidence; includes resolved citation

Generated example label: AI 생성 예문 (교재 원문 아님). Vocard example pairs must match the request exactly; known incorrect markers cannot become correct. Book examples must match retrieved extraction text, translation and correctness. Incorrect examples show a warning, not an affirmative correct-example badge.

Optional reference-book evidence

VOCARD_COURSE_BOOKS may explicitly map a vocabulary course to a reference book. Each request retrieves anew from that book's active index, using the word, current sentence and current question. It does not use the prior excerpt as retrieval evidence. No other configured book is a fallback.

The mapped reference book may have its own course metadata; vocabulary lookup scopes by authorized book ID rather than requiring the book's course ID to equal the vocabulary course. Grammar course/unit/lesson filters are unchanged. A mapped book with no active index returns 503.

Without a book mapping, tutoring still works with labeled Vocard context and model knowledge, but cannot cite book passages. Unknown citations, altered quotes and unattached book sources return 502. Book citations must support the explanation; exact-ID/quote checks establish provenance, not independent semantic verification.

Suggested and free-text follow-ups

Both send the current question, selected entry, current sentence and at most 2000 characters of the latest relevant explanation. The excerpt is untrusted, not a conversation or source citation. Previous questions, full exchanges and citation objects are not replayed.

Flutter displays only the latest tutor response. Changing the sentence resets response and prior context. Closing the screen or changing accounts clears temporary tutor state and discards late results. Opening performs no history/restore request. Vocabulary remains visible offline; AI answers require connectivity and are never restored from an answer cache.

Privacy, quota and provider retention

Application-level statelessness

Questions, answers and vocabulary temporary context exist only in request memory and Flutter screen memory. The online Q&A paths do not persist them in databases, files, caches, logs, analytics or traces. Authentication, private PDFs, extraction, indexes and operational metadata remain durable.

Only ai_daily_usage(uid, day, count) is stored for student Q&A. The shared grammar/vocabulary quota defaults to 50 admitted requests per verified UID per UTC day. Counters survive restarts; admitted provider failures and clarification requests count. Quota reservations are atomic. A 429 carries Retry-After: 3600, a hint rather than an exact reset countdown.

Both routes isolate LangChain/LangSmith tracing and inherited collectors, disable application generation caches, suppress SDK/access logging and sanitize validation/provider errors. Responses carry no-store headers. Configure gateways without message/body logging, disk buffering, response caching, session replay or content-capturing APM. Avoid message content in exception locals, span attributes and identifiers.

Model provider retention is separate

Generation uses ChatOpenAI(cache=False, store=False). This does not guarantee zero retention by the provider. The generation provider receives the selected context, including a temporary vocabulary excerpt; query embedding receives the query when retrieval is used. Provider logging, abuse monitoring, caching and approved retention controls have their own policies.

See docs/ai-privacy.md and OpenAI's data-controls guide: https://developers.openai.com/api/docs/guides/your-data . Review the actual model, endpoint and project controls before making a zero-retention claim. The separate Ollama/Gemini playground requires its own runtime/provider review.

Existing conversation data

Startup fails closed if legacy ai_conversations rows exist. Stop API/worker processes and explicitly authorize the documented migration before running it:

```
.venv/bin/python -m vocard.privacy_migration /path/vocard.sqlite3 \
  --confirm-delete-conversations
```

This opt-in operation removes the old conversation table, securely deletes SQLite records, checkpoints/truncates WAL and vacuums. It retains book/index data and usage counters. Historical backups, snapshots, copies and telemetry require separate cleanup; SQL deletion does not guarantee physical block erasure. Nothing is purged automatically.

Administrator retrieval and answers

POST /admin/books/{book_id}/search requires administrator book scope.

JSON field	Constraint / default
query	Required string, 1-10000 characters
top_k (alias k)	Optional strict integer 1-100; configured default 4
current_lesson	Optional strict integer >= 1
current_lesson_title	Optional nonblank string
course_id, unit_id	Optional nonblank exact scope filters
allow_book_fallback	Boolean; default true
max_fallback	Optional strict integer 0-100; configured default 2
min_score	Optional finite number -1 to 1; configured default 0

Returns {"documents":[...]} using LangChain Document serialization (page_content, metadata, id, type). Metadata includes book/version/page/lesson identity, retrieval scope and scores. Retrieve only from the authorized active index; lesson-first fallback stays bounded within book/course/unit scope.

POST /admin/books/{book_id}/answer

Accepts the search fields plus language (ko or en, default ko) and include_practice_examples (boolean, default false). Returns the shared grammar answer contract on page 05. history, message arrays and conversation IDs are rejected; the CLI no longer accepts --history-json.

```
.venv/bin/vocard ask grammar-book 'Explain gerunds as objects.' \
  --current-lesson 7 --no-book-fallback --top-k 4
```

GET /admin/books/{book_id}/lessons

Optional query filters: lesson_number >= 1, nonblank title, course_id, unit_id. limit defaults to 50 (range 1-200); offset defaults to 0 (>= 0). Exact source-order metadata lookup does not perform query embedding or fuzzy matching.

```
{"documents": [], "total_chunks": 0, "next_offset": null}
```

No filters returns scoped chunks. next_offset is null at the end. Search, lookup and answering return 404 when the book has no active index; authorization failures return 403. Invalid citations return 502; active-index changes return 409.

Administrator PDF ingestion

POST /admin/books/{book_id}/ingestions returns 202 after durable upload/enqueue. Run the worker separately; acceptance is not publication.

Multipart field	Contract
file	Required PDF; bytes start with %PDF-
extraction	Optional extraction JSON upload
reviewed_extraction	Boolean, default false; true requires extraction file
allow_uncertain	Boolean, default false; cannot bypass review
course_id, unit_id	Optional nonblank strings, at most 100 characters
chunk_size	Optional integer greater than overlap and <= 50000

```
curl "$BASE_URL/admin/books/devils-100/ingestions" \
  -H "Authorization: Bearer $ADMIN_TOKEN" \
  -F "file=@/path/book.pdf;type=application/pdf" \
  -F "extraction=@/path/reviewed.json;type=application/json" \
  -F "reviewed_extraction=true" -F "course_id=devils_100"
```

Returns the job record plus created (boolean), without lease_token. Important fields include job_id, book_id, sha256, book_version, status, stage, extraction_status, attempts and max attempts. Poll the job, then confirm the active index.

Limits and review

Default per-file limit is 100 MiB for both PDF and extraction; empty files are rejected. The total multipart cap is twice that limit plus 1 MiB. Oversized bodies return 413; input validation returns 422. Processing defaults cap PDFs at 2000 pages and indexes at 30000 chunks. Empty or suspicious extraction requires review rather than silent indexing. Korean/English OCR fallback preserves PDF indexes separately from printed labels.

Deduplication and publication

Source versions deduplicate by book and PDF SHA-256. Jobs deduplicate by source version and pipeline fingerprint, including extraction hash, review attestation and settings. Identical failed input returns its existing job. No Idempotency-Key header is implemented.

A durable worker extracts, chunks, embeds and validates an immutable private index. Only successful fenced publication changes the active pointer. Previous versions survive processing/failure; a late older job can be superseded instead of rolling back a newer publication.

Administrator jobs and review

GET /admin/jobs/{job_id}

Returns the job joined with book identity. Administrator job responses omit lease_token. Records are dictionaries; tolerate additive fields.

Field group	Included data
Identity	job_id, book_version_id, book_id, book_version, sha256, source_key
Lifecycle	status, stage, attempts, max_attempts, available_at, created_at, updated_at, lease_until, last_error
Pipeline/audit	sequence, pipeline_fingerprint, pipeline_json, requested_by, last_retried_by, publication_result
Review	extraction_key, extraction_status, book_extraction_status, extracted_key, extracted_sha256, review_status, review_report_json, review_report

Status is queued, running, succeeded or failed. Stage may indicate review_required. Timestamps are Unix seconds. Pipeline/review-report JSON fields are serialized strings; review_report is parsed JSON or null. Inspect publication_result to distinguish activation from supersession.

POST /admin/jobs/{job_id}/retry

No body is required. Only eligible failed jobs without review-required status can retry. Successful retry returns 202, records the administrator and grants three further attempts. Non-failed or review-required jobs return 409; unknown jobs return 404. Book authorization is checked.

GET /admin/jobs/{job_id}/extraction

Downloads saved extraction as application/json with Content-Disposition: attachment; filename="extraction.json". Unavailable jobs/extraction return 404. Compare the draft against the PDF, correct it, then resubmit the same source with the reviewed JSON and reviewed_extraction=true.

Reviewed imports must match the PDF hash and cover every PDF page exactly once in order. Empty pages require review_disposition="blank"; wholly empty books cannot publish. Embedded approval claims do not authorize review. Review-required jobs do not automatically retry or spend embedding calls.

GET /admin/books/{book_id}

Returns book_id, nullable active_index_version, created_at, versions and indexes. Versions list source hashes, extraction/review data in book-version order; indexes list immutable object/checksum/validation/job/previous-index history in sequence order. Unknown books return 404.

Errors and integration behavior

HTTP status	Meaning / client action
400	Malformed administrator Content-Length
401	Missing/invalid credentials; sign in or refresh token
403	Wrong role, book scope, anonymous user or unsupported/unavailable course
404	Unknown book/job, missing grammar index or unavailable extraction
409	Active index changed or administrator retry is ineligible
413	Administrator request exceeds body cap
422	Invalid/oversized input or forbidden extra fields
429	Shared student daily quota exhausted
502	Generated schema, source, citation or marker validation failed
503	Generation/auth unavailable; mapped vocabulary index unavailable
504	Standalone playground provider timeout

Sanitized Q&A errors

```
{"detail":"Invalid question request"}
{"detail":"Generated tutor response failed validation"}
{"detail":"AI service unavailable"}
```

Q&A validation failures return fixed messages rather than FastAPI input-echo details. Unexpected Q&A errors return a fixed 503 without request/provider traceback logging. Other non-Q&A routes may use normal FastAPI validation detail arrays. Never depend on raw model error content.

Client states and retries

Grammar `insufficient_evidence` and both routes' `clarification_needed` are valid 200 outcomes. Vocabulary `ambiguous` is also a successful result. Render these states explicitly rather than as network failures.

After a 409 active-index change, a new request retrieves current evidence. Do not blindly retry every 409: administrator job retry eligibility is a separate concern. The Flutter client refreshes one rejected Firebase token once; other failures require explicit retry. Offline, timeout, authentication, quota and unavailable-service states preserve only the open screen's draft.

There are no stored turns to recover after a timeout, no busy-conversation leases and no history endpoint. Retrying an admitted request can consume more quota. Responses are ordinary JSON, not SSE. No HTTP endpoints implement conversation deletion, token minting, job cancellation, book deletion or manual index rollback.

Separate playground API

App factory: vocard.chat:create_chat_app. Default provider is local Ollama, with an optional hosted Gemini mode. This app does not use the Firebase production API or its authorized vector index.

```
.venv/bin/uvicorn vocard.chat:create_chat_app --factory \
  --host 127.0.0.1 --port 8001 --no-access-log
```

Route	Result
GET /healthz	Public process health
GET /	HTML playground UI
GET /api/parts	parts array with id, title and page_count
GET /api/parts/{part_id}	id, title and PDF-page text
POST /api/chat	answer string

```
{"part_id":"part-1", "question":"Explain gerunds."}
```

part_id is optional/null or a string of at most 100 characters. question is required, nonblank, 1-10000 characters. This route accepts one independent question; messages and conversation history are rejected. Q&A responses/errors have no-store behavior. Browser display state is temporary; it is not replayed in requests.

When a part is selected, reference excerpts come from that part within a 14000-character budget. The source is an unreviewed OCR draft, not the production index. Unknown part IDs return 404; unavailable grammar content returns 503. The answer is free-form text and does not use production citation validation.

Ollama is called at loopback port 11434 with stream=false, a 180-second timeout and default model qwen2.5:3b. Optional Gemini uses VOCARD_CHAT_PROVIDER=gemini, GEMINI_API_KEY and VOCARD_GEMINI_MODEL (default gemini-2.5-flash-lite), with a 90-second timeout.

Local mode without configured credentials is unauthenticated and must remain loopback-only. Hosted/Gemini mode requires HTTP Basic username and a password of at least 16 characters. Health remains public; other routes require credentials when authentication is configured. Connection failures return 503, timeout 504, quota 429 and other provider failures 502. Keep this prototype separate from production tutoring; provider retention is independent.

Configuration and source map

Variable	Default / use
VOCARD_S3_BUCKET / VOCARD_JWT_SECRET	Required private bucket / secret 〉= 32 bytes
VOCARD_DB_PATH	state/ingestion.sqlite3
VOCARD_FIREBASE_PROJECT_ID	Required to verify Firebase student tokens
VOCARD_COURSE_BOOKS	{}; JSON course-to-authorized-book mapping
VOCARD_AI_DAILY_QUOTA	50 shared grammar/vocabulary requests per UID per UTC day
VOCARD_TOP_K / VOCARD_MAX_FALLBACK	4 / 2
VOCARD_MIN_SCORE	0
VOCARD_ANSWER_MODEL	gpt-4.1-mini; grammar and vocabulary generation
VOCARD_ANSWER_TIMEOUT	60 seconds per attempt; one provider retry
VOCARD_ANSWER_MAX_TOKENS	3000
VOCARD_ANSWER_CONTEXT_CHARS	60000 serialized context characters
VOCARD_CHUNK_SIZE / VOCARD_CHUNK_OVERLAP	1000 / 150
VOCARD_CHAT_MODEL	qwen2.5:3b; standalone Ollama only
VOCARD_GRAMMAR_PATH	output/pdf/textbook-extraction/textbook.json

VOCARD_AI_HISTORY_MESSAGES and VOCARD_AI_HISTORY_CHARS are retired. They no longer configure retention. OPENAI_API_KEY enables embeddings and generation. AWS uses the normal boto3 credential chain; Firebase uses server Application Default Credentials. No provider keys belong in Flutter. Its HTTPS base URL is configured with VOCARD_AI_BASE_URL.

Repository references

- Production routes: vocard/api.py, vocard/student_api.py; authentication: vocard/auth.py, vocard/student_auth.py.
- Answering and vocabulary: vocard/answering.py, vocard/vocabulary.py; contracts: schema/answer-response.schema.json, schema/vocabulary-question.schema.json, schema/vocabulary-tutor-response.schema.json.
- Privacy and quota: vocard/privacy.py, vocard/usage.py, vocard/privacy_migration.py, docs/ai-privacy.md.
- Ingestion/retrieval: vocard/ingestion.py, vocard/db.py, vocard/worker.py, vocard/retrieval.py; extraction contract: schema/grammar-extraction.schema.json.
- Detailed vocabulary contract: docs/vocabulary-tutor-api.md; operations: README.md; source for this PDF: docs/api-reference.md.

Verification: repository routes/models and documentation reviewed on 17 October 2026. Automated tests use mocked providers; live model linguistic accuracy and deployment readiness are not established by this reference. Update this Markdown source and regenerate the PDF whenever an API contract changes.